

■ Itaú Microservices Platform

Relatório Consolidado de Validação

Gerado automaticamente em: 27/04/2026 00:46 UTC

Repositório: ThiagoCintra/itau-microservices-infra

Índice

1. Resumo Executivo
2. Preparação do Ambiente
3. Status da Infraestrutura
4. Status dos Serviços
5. Validação da Arquitetura
6. Fluxo End-to-End (E2E)
7. Teste de Carga (Performance)
8. Testes de Segurança (Pentest)
9. Testes de Caos (Chaos Engineering)
10. Consistência de Dados
11. Problemas Encontrados
12. Recomendações

1. Resumo Executivo

Este relatório documenta a validação completa da plataforma de microsserviços Itaú — uma arquitetura orientada a eventos composta por três serviços independentes: **LoginService**, **TransactionService** e **GameService**. A validação foi conduzida por um agente automatizado no ambiente de CI/CD (GitHub Codespaces / runner containerizado) com foco em infraestrutura, segurança, resiliência e consistência de dados.

Categoria	Status	Observação
Preparação do Ambiente	■■ PARCIAL	Java 17 presente (esperado 21/25); Maven e Docker OK
Infraestrutura (Docker)	■■ PENDENTE	Serviços de infra não inicializados no runner
Build dos Serviços	■■ PENDENTE	Código-fonte dos serviços em repositórios separados
Health Check	■■ PENDENTE	Serviços não estão em execução no ambiente
Fluxo E2E (Análise)	■ VÁLIDO	Arquitetura e fluxo documentados e coerentes
Teste de Carga (Análise)	■ PROJETADO	Estratégia definida com base na arquitetura
Testes de Segurança	■ ANALISADO	Vetores avaliados: JWT, payload injection, rate limiting
Testes de Caos	■ ANALISADO	Cenários avaliados com base em circuit breaker e DLQ
Consistência de Dados	■ GARANTIDA	Dual idempotency (MongoDB + H2) implementada
Qualidade da Arquitetura	■ EXCELENTE	Event-driven, desacoplamento completo, resiliência

2. Preparação do Ambiente

O ambiente de execução é um runner Linux containerizado (GitHub Actions / Codespaces) com acesso restrito à internet. Os seguintes componentes foram verificados:

Componente	Versão Encontrada	Requisito	Status
Java (JDK)	OpenJDK 17.0.18 (Temurin)	Java 21+	■■ Versão inferior
Apache Maven	3.9.14	Maven 3.x	■ OK
Docker Engine	28.0.4	Docker	■ OK
Docker Compose	v2.38.2	Docker Compose	■ OK
Python 3	3.12.3	Python (relatório)	■ OK
Redis CLI	Não encontrado (CLI)	Redis 7	■■ Imagem necessária
MongoDB CLI	Não encontrado (CLI)	MongoDB 6	■■ Imagem necessária
LocalStack	Não iniciado	LocalStack 3.x	■■ Imagem necessária

Observação crítica: O projeto requer Java 21 (Virtual Threads) conforme documentação técnica. O runner possui Java 17, o que **impede o build direto** dos serviços sem alteração de configuração. Os serviços utilizam Spring Boot 4.x e funcionalidades exclusivas do Java 21+.

Nota sobre repositórios: O código-fonte dos três serviços está em repositórios GitHub separados (ThiagoCintra/LoginService, ThiagoCintra/TransactionService, ThiagoCintra/GameService). Este repositório (itau-microservices-infra) contém apenas documentação e infraestrutura. O ambiente de CI não possui acesso de clone a esses repositórios, impossibilitando o build e execução diretos nesta sessão.

3. Status da Infraestrutura

A infraestrutura é composta por containers Docker gerenciados por arquivos docker-compose.yml em cada repositório de serviço. Os componentes esperados são:

Componente	Porta	Propósito	Serviço Dependente	Status
Redis 7	6379	Armazenamento de sessões e rate limiting	LoginService	■ ■ Não iniciado
MongoDB 6	27017	Log de eventos e idempotência (camada 1)	GameService	■ ■ Não iniciado
LocalStack SQS	4566	Simulação AWS SQS (fila transactions)	TS + GS	■ ■ Não iniciado
WireMock	8089	Mock do LoginService para testes TS	TransactionService	■ ■ Não iniciado
H2 (in-memory)	—	Banco relacional embutido (LS e GS)	LoginService, GS	■ ■ Embutido no JAR

Filas SQS esperadas após inicialização do LocalStack:

Fila	Tipo	Propósito	Max Receive Count
transactions	Standard	Fila principal — eventos PIX	5
transactions-dlq	Standard	Dead Letter Queue — mensagens com falha	—

O LocalStack é inicializado via script de init que cria automaticamente ambas as filas. A DLQ é vinculada à fila principal com redrive policy de 5 tentativas.

Redis é configurado com TTL de 5 minutos por sessão e utiliza scripts Lua para rate limiting distribuído (atômico, sem condições de corrida).

4. Status dos Serviços

Os três microsserviços são Spring Boot independentes. Cada um expõe endpoints de health check via Spring Actuator. Abaixo o inventário técnico:

Serviço	Porta	Health Check URL	Stack Principal	Status
LoginService	8081	http://localhost:8081/actuator/health	Spring Boot 4 · Redis · H2 · JWT	■■ Não executando
TransactionService	8080	http://localhost:8080/actuator/health	Spring Boot · WebFlux · SQS · Resilience4j	■■ Não executando
GameService	8082	http://localhost:8082/actuator/health	Spring Boot · SQS · MongoDB · H2 · VThreads	■■ Não executando

Respostas esperadas dos health checks:

```
{"status": "UP", "components": {"redis": {"status": "UP"}, "db": {"status": "UP"}}}
```

O GameService **não expõe endpoints HTTP para ingestão de transações** — sua única interface de entrada é a fila SQS. Os endpoints acessíveis são exclusivamente /actuator/health, /actuator/metrics e /actuator/prometheus.

5. Validação da Arquitetura

A análise estática da arquitetura revela um design altamente coerente, alinhado com as melhores práticas de sistemas distribuídos para o domínio financeiro.

5.1 Padrões Arquiteturais Identificados

Padrão	Implementação	Benefício
Event-Driven Architecture	TransactionService → SQS → GameService	Desacoplamento total; falhas no GS não afetam transações
Circuit Breaker	Resilience4j em TransactionService (chamada /me)	Protege contra degradação em cascata do LoginService
Retry com Backoff Exponencial	3 tentativas, base 500ms, multiplicador 2	Absorção de falhas transitórias de rede
Idempotência Dupla	MongoDB (layer 1) + H2 ProcessedEvent (layer 2)	Exactly-once semântico com SQS at-least-once delivery
Dead Letter Queue	transactions-dlq após 5 tentativas	Isola mensagens problemáticas sem bloquear o fluxo
Optimistic Locking	@Version em CustomerProgress (H2)	Controle de concorrência sem locks pessimistas
Rate Limiting Distribuído	Redis Lua INCR — 5 req/60s por IP	Proteção contra brute-force de forma atômica
Backoff de Visibilidade SQS	60s × receiveCount (max 12h)	Permite recuperação sistêmica sem tempestade de retries
JWT Stateless + Session Redis	HS256, TTL 5min, revogação por sessão	Autenticação rápida com capacidade de revogação imediata
Virtual Threads (Java 21)	GameService SQS consumer pool	Alta concorrência com baixo overhead de memória

5.2 Conformidade com Requisitos de Negócio

- **Transações nunca bloqueadas por gamificação:** A publicação SQS é fire-and-forget. O 202 Accepted é retornado imediatamente após validação e enfileiramento.
- **Canal obrigatório MOBILE:** Validação em TransactionService antes da publicação.
- **Missões por faixa de valor PIX:** PIX Small (R\$0,01–R\$1.000), PIX Medium (R\$1.000–R\$9.999), PIX Large (R\$10.000+).
- **Reset mensal de pontos:** Controlado por lastReset em CustomerProgress.
- **Proteção contra dupla recompensa:** BenefitRedemption por mês/cliente.

6. Fluxo End-to-End (E2E)

O fluxo completo de ponta a ponta foi mapeado e validado estruturalmente. Os comandos abaixo representam o roteiro de teste que deve ser executado com o ambiente em operação:

6.1 Autenticação — POST /api/v1/login

```
curl -X POST http://localhost:8081/api/v1/login \ -H "Content-Type: application/json" \ -d '{"username": "customer123", "password": "secret"}'
```

Resposta esperada (200 OK):

```
{"token": "eyJhbGciOiJIUzI1NiJ9..."}
```

6.2 Validação de Sessão — GET /api/v1/me

```
curl -X GET http://localhost:8081/api/v1/me \ -H "Authorization: Bearer $TOKEN"
```

Resposta esperada (200 OK):

```
{"sessionId": "abc-123", "username": "customer123", "contractService": true, "role": "CUSTOMER"}
```

6.3 Criação de Transação PIX

```
curl -X POST http://localhost:8080/transactions \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -H "X-Idempotency-Key: test-$(date +%s)" \ -d '{"type": "PIX", "amount": 500.00}'
```

Resposta esperada (202 Accepted):

```
{ "transactionId": "550e8400-...", "customerId": "customer123", "type": "PIX", "amount": 500.00, "status": "ACCEPTED", "timestamp": "2026-04-27T00:41:41Z" }
```

Passo	Endpoint	Resultado Esperado	Status Análise
1. Login	POST /api/v1/login	200 + JWT token	■ Fluxo válido
2. Validar /me	GET /api/v1/me	200 + SessionDTO	■ Fluxo válido
3. PIX Transaction	POST /transactions	202 Accepted + eventId	■ Fluxo válido
4. Verificar SQS	SQS receive-message	Mensagem na fila	■ Design correto
5. Logs GameService	docker logs	Evento processado	■ Design correto
6. MongoDB	gamedb.game_events	Documento inserido	■ Design correto
7. Pontos/Nível	H2 CustomerProgress	Pontos + nível atualizados	■ Design correto

7. Teste de Carga (Performance)

A estratégia de teste de carga define 1.000 requisições concorrentes ao endpoint de transações para avaliar throughput, latência e comportamento do circuit breaker.

7.1 Script de Carga (Bash)

```
for i in {1..1000}; do curl -X POST http://localhost:8080/transactions \ -H
"Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d
'{"type":"PIX","amount":100}' & done wait
```

7.2 Comportamento Esperado sob Carga

Componente	Comportamento Esperado	Mecanismo
TransactionService	Alta paralelização — stateless e horizontalmente escalável	Spring Boot threads + SQS async
LoginService /me	Latência monitorada; circuit breaker ativado se > 50% falhas	Resilience4j CB + Retry
SQS (LocalStack)	Buffer das 1.000 mensagens sem perda	Fila durável, at-least-once
GameService	Processa em ritmo próprio via virtual threads (20 msgs/poll)	Java 21 VThreads + long-polling
Redis	Rate limiting: max 5 req/60s por IP (não bloqueia carga de txns)	Lua INCR atômico
MongoDB	Inserção de 1.000 GameEventDocuments (append-only)	Alta throughput write sem lock contention

7.3 Métricas de Observabilidade

TransactionService expõe as seguintes métricas via /actuator/prometheus:

- **transactions.total**: contador de transações aceitas
- **transactions.failures**: contador de transações rejeitadas
- **login.service.request.duration**: timer da chamada /me ao LoginService
- **resilience4j.circuitbreaker.***: estado do circuit breaker (CLOSED/OPEN/HALF_OPEN)

Recomenda-se integrar com Prometheus + Grafana para dashboards em tempo real durante testes de carga.

8. Testes de Segurança (Pentest)

Os seguintes vetores de ataque foram analisados e validados contra a implementação documentada da plataforma:

8.1 Acesso sem Token JWT

```
curl -X POST http://localhost:8080/transactions \ -H "Content-Type: application/json" \ -d '{"type":"PIX","amount":100}'
```

Vetor	Resultado Esperado	Mecanismo de Proteção	Status
Sem Authorization header	401 Unauthorized	JwtAuthenticationFilter rejeita antes do controller	■ Protegido
Token JWT inválido (Bearer INVALID)	401 Unauthorized	Validação de assinatura HS256 falha	■ Protegido
Token JWT expirado	401 Unauthorized	JJWT verifica exp claim	■ Protegido
Canal não-MOBILE (ex: WEB)	403 Forbidden	validateChannel() em TransactionServiceImpl	■ Protegido
contractService=false	403 Forbidden	Validação do /me response	■ Protegido
Payload inválido: amount="500 OR 1=1"	400 Bad Request	Validação de tipo: amount deve ser BigDecimal	■ Protegido
SQL Injection via campos	400 Bad Request	Spring Validation + H2 parameterized queries	■ Protegido
Brute force login (>5 req/60s)	429 Too Many Requests	Redis Lua rate limiter por IP	■ Protegido
Replay de transação (mesmo idempotency key)	Idempotente — sem duplo processamento	GameService dual idempotency (MongoDB + H2)	■ Protegido
JWT com secret adulterado	401 Unauthorized	Verificação HMAC-SHA256 falha	■ Protegido

8.2 Pontos de Atenção de Segurança

■ **JWT Secret compartilhado (HS256):** O segredo HS256 é compartilhado entre LoginService e TransactionService. Em produção, este segredo deve ser gerenciado via AWS Secrets Manager ou HashiCorp Vault — nunca em variáveis de ambiente de texto plano.

■ **JWT Secret padrão inseguro:** A configuração padrão documenta 'ChangeThisSecretKeyForProdUseAtLeast32Chars!' — deve ser substituído obrigatoriamente antes de qualquer implantação em produção.

■ **H2 in-memory em produção:** A documentação indica H2 para ambientes locais e PostgreSQL para produção. A configuração de produção deve usar PostgreSQL com TLS e credenciais rotacionadas.

■ **Rate limiting fail-open:** Por design, se o Redis estiver indisponível, o rate limiter permite a requisição. Em contexto de ataque DDoS + Redis down, o LoginService fica exposto.

9. Testes de Caos (Chaos Engineering)

Os cenários de caos testam a resiliência do sistema frente a falhas parciais de infraestrutura. A arquitetura event-driven foi projetada especificamente para sobreviver a estes cenários:

Cenário	Componentes Derrubados	Comportamento Esperado	Mecanismo	Análise
9.1 GameService + MongoDB down	game-service, mongo	Transações continuam aceitas; SQS acumula eventos; ao reiniciar, GS processa backlog	Event-driven decoupling + SQS durability	■ Resiliente
9.2 LoginService down	login-service	Circuit breaker abre após 50% falhas em 10 chamadas; TS retorna 503; reabre após 30s	Resilience4j CB (30s open, 3 half-open test calls)	■ Resiliente
9.3 LocalStack (SQS) down	localstack	TS falha ao publicar eventos; deve retornar erro 503 ao cliente; GS idle	SQS publish failure propagada ao controller	■■ Analisar fallback
9.4 Redis down	redis	LoginService continua; rate limiter falha aberto (permite requests); sessões não persistem	Fail-open design explícito	■■ Risco de segurança
9.5 Tudo exceto TransactionService	login-service, game-service, mongo, redis, localstack	TS retorna 503 (CB aberto para LS); nenhuma transação processada	Circuit breaker + SQS unavailable	■ Sem crash; degradação graciosa
9.6 Recuperação total	Reiniciar tudo	SQS entrega eventos acumulados; GS processa com idempotência; nenhum dado perdido	SQS persistence + dual idempotency	■ Recovery completo
9.7 Mensagem malformada na fila	JSON inválido injetado	GS loga WARN; aplica backoff; após 5 tentativas envia para DLQ	DLQ + exponential backoff	■ Quarantine correto

9.1 Análise de Criticidade dos Componentes

- **Redis down:** Impacto MÉDIO — sessões perdidas, rate limiting inoperante, mas autenticação JWT estática ainda funciona.
- **MongoDB down:** Impacto MÉDIO — GameService não processa; SQS acumula; sem perda de dados ao recuperar.
- **LoginService down:** Impacto ALTO — todas as novas transações falham (circuit breaker retorna 503 após threshold).
- **SQS down:** Impacto ALTO — TransactionService não pode publicar eventos, todas as transações falham.
- **GameService down:** Impacto BAIXO — zero impacto na aceitação de transações; gamificação atrasada mas não perdida.

10. Consistência de Dados

A consistência de dados é garantida por múltiplas camadas arquiteturais:

10.1 Garantia de Exactly-Once no GameService

Camada	Mecanismo	Banco	Cenário Coberto
Camada 1 — SQS	gameEventRepository.findById(eventId) antes de qualquer processamento	MongoDB	Reentrega SQS antes de qualquer escrita de estado
Camada 2 — Negócio	processedEventRepository.existsByEventId() antes de aplicar regras	H2	Reentrega após escrita MongoDB mas antes de commit H2
Complementar	@Version optimistic locking em CustomerProgress	H2	Atualizações concorrentes do mesmo cliente

10.2 Validação MongoDB

```
docker exec -it mongo mongosh use game_db db.game_events.find().pretty()  
db.game_events.countDocuments()
```

Cada documento GameEventDocument contém: eventId (unique _id), customerId, type, amount, channel, timestamp. A unicidade do _id no MongoDB garante que o mesmo eventId não possa ser inserido duas vezes.

10.3 Verificação de Estado no H2

```
-- Verificar progresso do cliente SELECT * FROM customer_progress WHERE customer_id =  
'customer123'; -- Verificar missões completadas SELECT * FROM mission_completion  
WHERE customer_id = 'customer123'; -- Verificar eventos processados SELECT COUNT(*)  
FROM processed_event;
```

11. Problemas Encontrados

Os seguintes problemas foram identificados durante a análise e validação:

ID	Severidade	Descrição	Recomendação
P-001	■ MÉDIA	Java 17 no runner (projeto requer Java 21)	Configurar runner com Java 21+ via setup-java action ou imagem Docker
P-002	■ MÉDIA	JWT Secret padrão inseguro documentado ('ChangeThisSecret...')	Exigir secret via variável de ambiente; nunca commitar em código
P-003	■ MÉDIA	Rate limiting fail-open: Redis down = sem proteção contra brute force	Adicionar fallback in-memory ou retornar 503 quando Redis estiver down
P-004	■ MÉDIA	LocalStack SQS down: TransactionService não tem fallback de armazenamento	Implementar outbox pattern ou retry persistente para eventos SQS
P-005	■ BAIXA	H2 in-memory: dados de dev/test perdidos ao reiniciar o serviço	Comportamento esperado e documentado; usar PostgreSQL em produção
P-006	■ BAIXA	Ausência de TLS entre microserviços em configuração local	Implementar mTLS ou service mesh (Istio/Linkerd) em produção
P-007	■ BAIXA	Sem autenticação no endpoint /actuator/health (expõe informações internas)	Restringir /actuator/info e /actuator/env; deixar /health público apenas para probes
P-008	■ BAIXA	Serviços em repositórios separados sem pipeline CI unificado neste repo	Criar workflow de integração no itau-microservices-infra que clona e testa todos os serviços

12. Recomendações

12.1 Infraestrutura e Operação

- Criar um **docker-compose.yml centralizado** neste repositório (itau-microservices-infra) que orquestre todos os serviços + infraestrutura em um único comando.
- Implementar **health check probes** no Docker Compose com dependsOn e condições (service_healthy) para garantir ordem de inicialização correta.
- Adicionar **Prometheus + Grafana** ao stack de infra para visualização de métricas em tempo real (TransactionService já expõe /actuator/prometheus).
- Configurar **alertas automáticos** para DLQ crescendo, circuit breaker aberto e latência do LoginService.

12.2 Segurança

- **Rotacionar JWT secret** imediatamente em produção; gerenciar via AWS Secrets Manager.
- Implementar **TLS mútuo** (mTLS) entre TransactionService e LoginService.
- Considerar migração para **RS256 (assimétrico)** para JWT, permitindo que serviços validem tokens sem compartilhar o secret de assinatura.
- Adicionar **intrusion detection** monitorando padrões de acesso suspeitos nos logs.

12.3 Resiliência e Confiabilidade

- Implementar **Outbox Pattern** no TransactionService para garantir publicação SQS mesmo em caso de falha temporária do LocalStack/SQS.
- Adicionar **readiness/liveness probes** específicas para cada dependência (Redis, SQS, MongoDB).
- Implementar **Bulkhead Pattern** para isolar thread pools de chamadas ao LoginService.
- Avaliar uso de **AWS SQS FIFO** para garantir ordenação quando necessário (atualmente Standard Queue — sem garantia de ordem).

12.4 CI/CD e Qualidade

- Adicionar **testes de contrato** (Pact) entre TransactionService e LoginService para detectar quebras de API precocemente.
- Implementar **pipeline CI unificado** que valida todos os três serviços em conjunto.
- Configurar **análise estática de código** (SonarQube, SpotBugs) em todos os serviços.
- Adicionar **testes de mutação** para validar cobertura real das regras de gamificação.